LLVM DEV MEETING 2026 · MOJO WORKSHOP

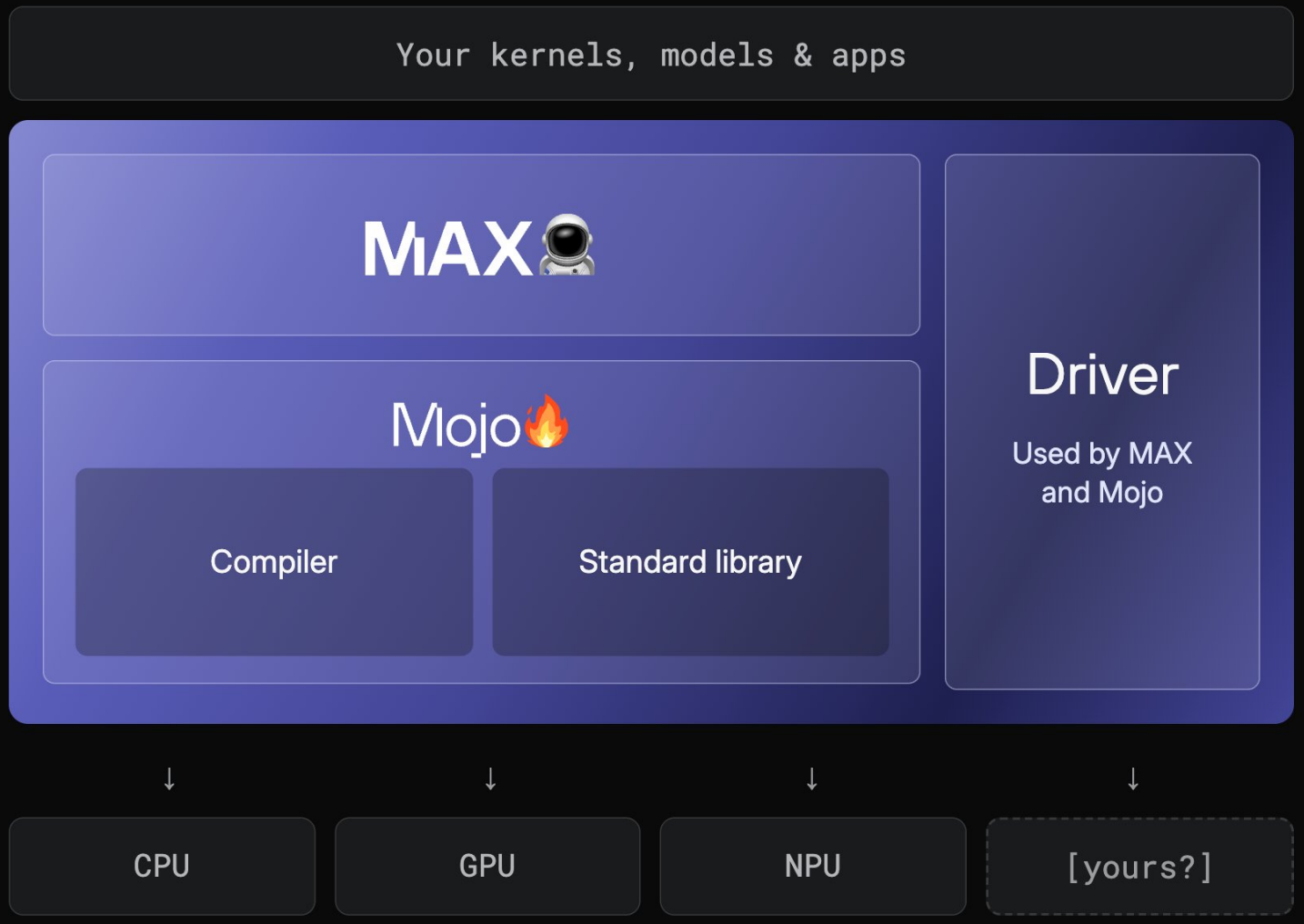
Making new hardware boring with Modular Plugins

Connor Gray

Part 1 of 2, with Manoj Rajagopalan

Modular

The Modular stack



Portable, performant, programmable AI-native software stack.

Result of 4+ years of building from the ground up.

Today: runs on CPUs and accelerators from 6+ vendors.

AI developers can...

1 Performance

Author kernels that get speed-of-light performance

No optimization should be impossible or difficult due to barriers or costly abstractions.

on every accelerator they target

2 Portability

Write against common primitives and algorithms

~~`vendor_intrinsic_cos(x)`~~

`cos(x)`

You should never have to write a vendor intrinsic. Call the standard Mojo **cos()** and know you're getting the best performance.

AI developers can...

1 Performance

Author kernels that get speed-of-light performance

on every accelerator they target

2 Portability

Write against common primitives and algorithms

~~`vendor_intrinsic_cos(x)`~~

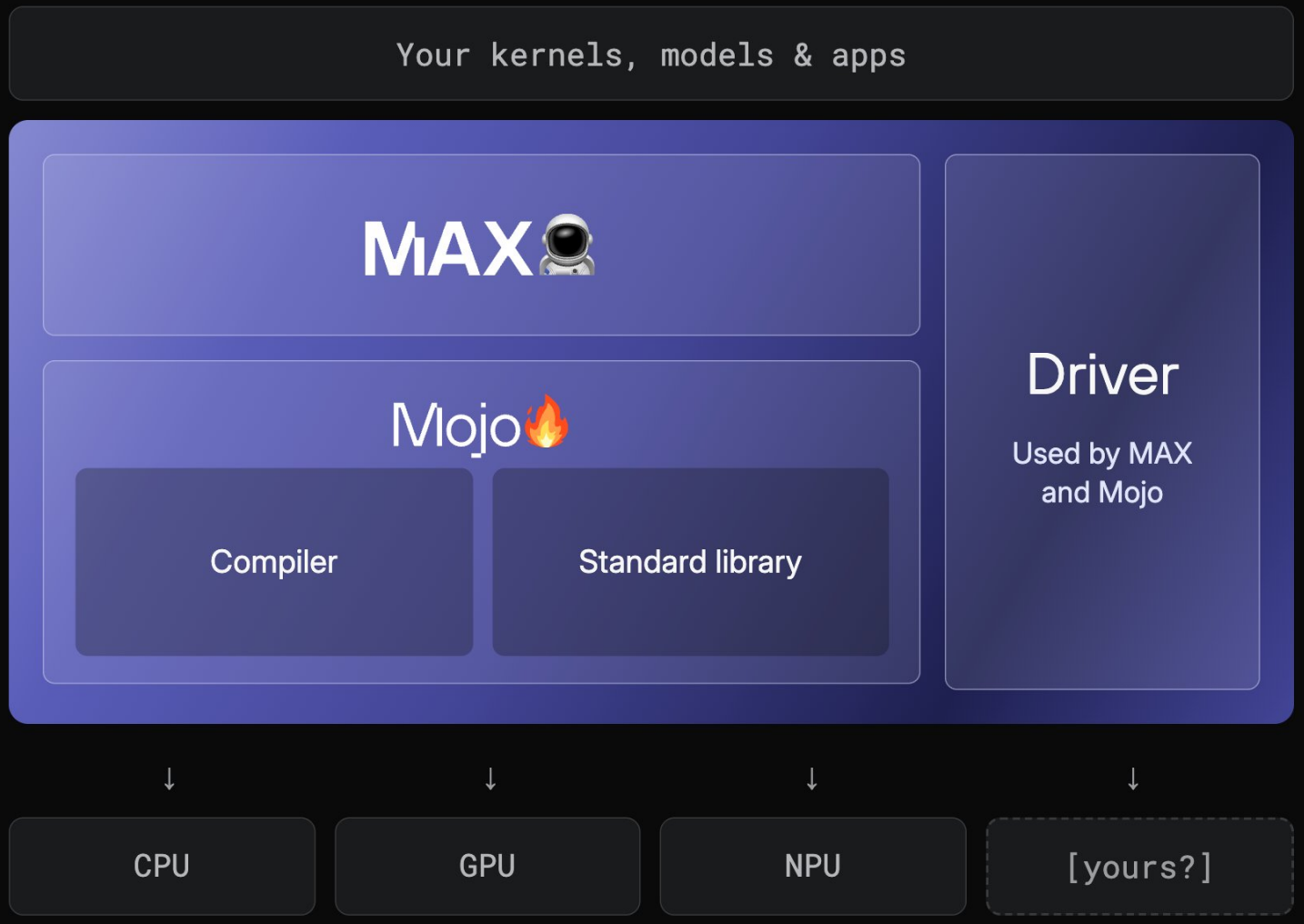
`cos(x)`

...which requires...

3 Programmability

Teach every layer of the stack about new hardware, without forking it

The Modular stack

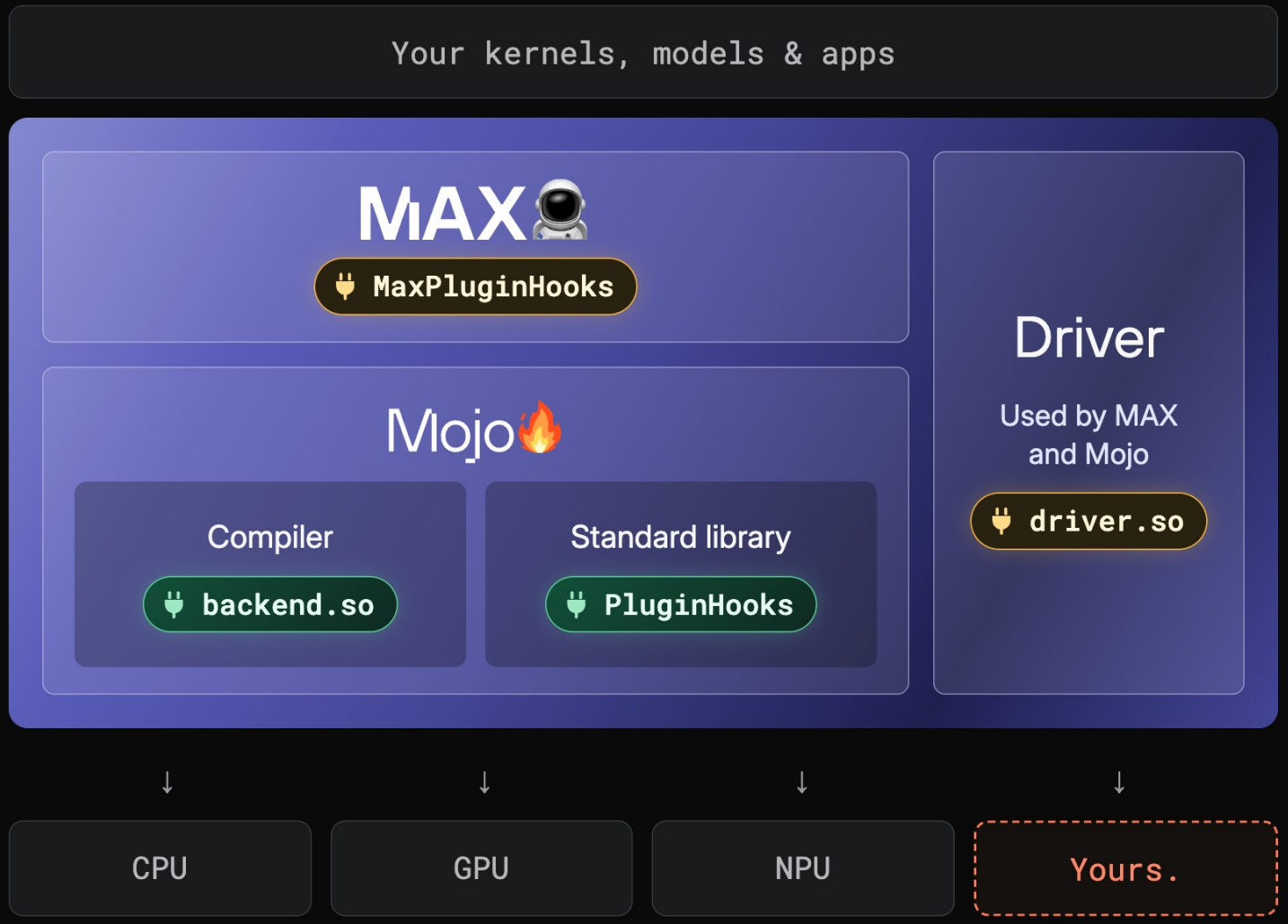


Portable, performant, programmable AI-native software stack.

Result of 4+ years of building from the ground up.

Today: runs on CPUs and accelerators from 6+ vendors.

Every layer, now extensible by anyone



The Modular stack has long been heterogeneous across hardware. Now it's extensible by anyone.

`plug` = an extension point

Two Mojo traits, two dynamic libraries

`BUILT` Compiler & stdlib plugins

`IN PROGRESS` Driver & MAX plugins

Shared plug points let vendors and contributors improve shared components of a common stack, together, instead of each keeping a fork.

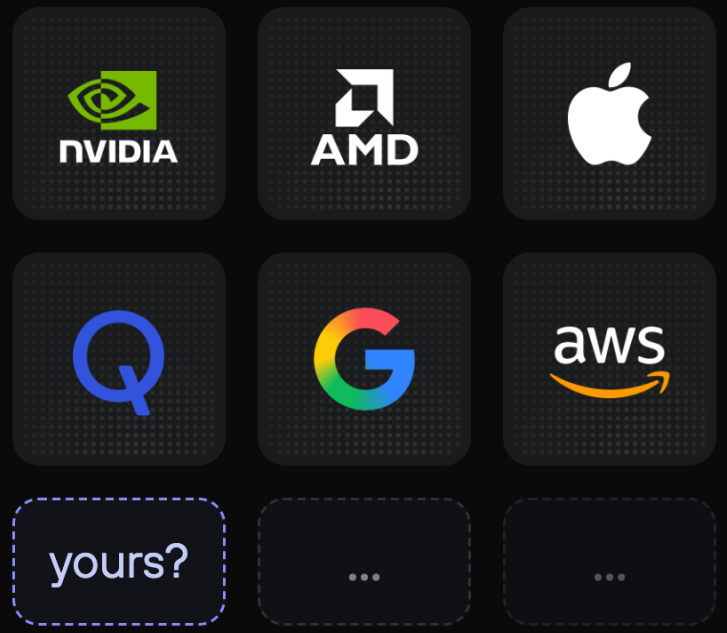
modular.com/alliance

Uses author one Mojo source. Portable across vendors.

```mojo
def cos_kernel(x: TileTensor, out: TileTensor):
    var i = global_idx.x
    out[i] = cos(x[i])

def main():
    var ctx = DeviceContext()

    ctx.enqueue_function[cos_kernel](
        x, out, grid_dim=n // 256, block_dim=256
    )
```



This example uses only standard library, MAX primitives, and MAX driver APIs, and works across vendors without changes.

Previously: Hardware knowledge was hard-coded in the stdlib

```mojo
def cos(x: SIMD) -> type_of(x):
    comptime if size_of[dtype]() < size_of[DType.float32]():
        return cos(x.cast[.float32]()).cast[dtype]()

    comptime if is_nvidia_gpu() and dtype == .float32:
        return _call_ptx_intrinsic[
            instruction="cos.approx.ftz.f32", constraints="=f,f"
        ](x)
    elif is_apple_gpu():
        return _llvm_unary_fn["llvm.air.cos"](x)
    else:
        return _llvm_unary_fn["llvm.cos"](x)
```

A closed set of is_*() checks

- Benefit: vendor implementation is easy to find. Just read the code.
- Downside: to add a vendor, you edit the standard library itself.

The previous solution leveraged Mojo's powerful zero-cost metaprogramming system to select the optimized implementation at compile time.

Today: cos() asks the current plugin first

```
def cos(x: SIMD) -> type_of(x):
    if __is_run_in_comptime_interpreter:
        return _llvm_unary_fn["llvm.cos"](x)

    comptime if case .Some(lower) = CurrentPlugin.unary_math_fn[
        .cos, dtype, width
    ]:
        return lower(x)

    comptime if size_of[dtype]() < size_of[DType.float32]():
        return cos(x.cast[.float32]()).cast[dtype]()
    return _llvm_unary_fn["llvm.cos"](x)
```

The new standard library implementation factors out shared boilerplate logic, but invokes the plugin for the optimized core implementation.

But what is CurrentPlugin?

```
comptime
CurrentPlugin:
  PluginHooks
  = ...?
```

We'll come back to this in 3 slides →

Stdlib vendor code fully moved into plugins

`cuda/cuda_math.mojo`

`hip/hip_math.mojo`

`metal/metal_plugin.mojo`

`qaic_std/backend.mojo`

PluginHooks: a trait of optional compile-time hooks

```
trait PluginHooks:
    comptime unary_math_fn[
        op: _UnaryMathOp, dtype: DType, width: Int
    ]: OptionalReg[_UnaryMathPluginHookFnType] = None

    comptime cast_fn[src: DType, dst: DType]: OptionalReg[
        _CastPluginHookFnType[dst]
    ] = None

    comptime stack_allocation_fn[address_space: AddressSpace]: ...
    comptime print_emit_fn: OptionalReg[_PrintEmitPluginHookFnType] = None
    ...  # 24 hooks today
```

Default: None

The stdlib uses its own default lowering

All comptime

No runtime dispatch, zero cost

New hooks added as needed

Math, casts, atomics, memory, thread ids, printing

Implementing plugin hooks: a whole accelerator's stdlib as a plugin

```
struct QAICBackendHooks[target: CompilationTarget](PluginHooks):
    # exp and tanh on HVX vectors, via the Hexagon runtime
    comptime unary_math_fn[op, dtype, width] =
        _qaic_unary_math_fn[op, dtype, width]()
    # SHARED memory lives in on-chip VTCM
    comptime stack_allocation_fn[address_space] =
        _stack_allocation[...] if address_space == .SHARED else None
    comptime print_emit_fn =
        _print_emit
    comptime reduce_generator_fn =
        _reduce_generator_npu
```



263 lines

Supports Cloud AI 100 onwards. Every unused hook uses the stdlib default.

The plugin is just another field of the target

① So what is CurrentPlugin? It comes from the current target.

```
comptime CurrentPlugin: PluginHooks = CompilationTarget.current().stdlib_plugin
```

② And a Mojo target carries rich behavior, not just plain data.

```
comptime _qaic_aic100_target = CompilationTarget[
    _mlir_value=__mlir_attr[
        `#kgen.target<triple = "hexagon-unknown-linux-musl", `,      ← data
        `arch = "hexagonv68", features = "+hvx,...", `,              ← data
        `simd_bit_width = 1024, `,                                   ← data
        `index_bit_width = 32, `,                                    ← data
        `opaque_plugin = `, TypeList.of[QAICBackendHooks]().values,  ← behavior
        `> : !kgen.target`,
    ]
]()
```

Credit: Sawyer Bergeron · building on Steffi Stumpos and Billy Zhu

Your own plugin, in a dozen lines

my_plugin.mojo

```
def my_cos(x: SIMD) -> type_of(x):
    return __mlir_op.`pop.call_llvm_intrinsic`[  # drop straight to MLIR
        intrin="llvm.my_device.cos", _type=type_of(x)
    ](x)

struct MyPluginHooks[target: CompilationTarget](PluginHooks):
    comptime name = "my_device"
    comptime unary_math_fn[op: _UnaryMathOp, dtype: DType, width: Int] =
        OptionalReg(my_cos) if op == .cos else None

comptime MY_DEVICE = CompilationTarget[#kgen.target<..., opaque_plugin = [MyPluginHooks]>]
```

user_code.mojo

```
var f = compile_info[my_kernel, target=MY_DEVICE]()   # cos() now lowers to my_cos
```

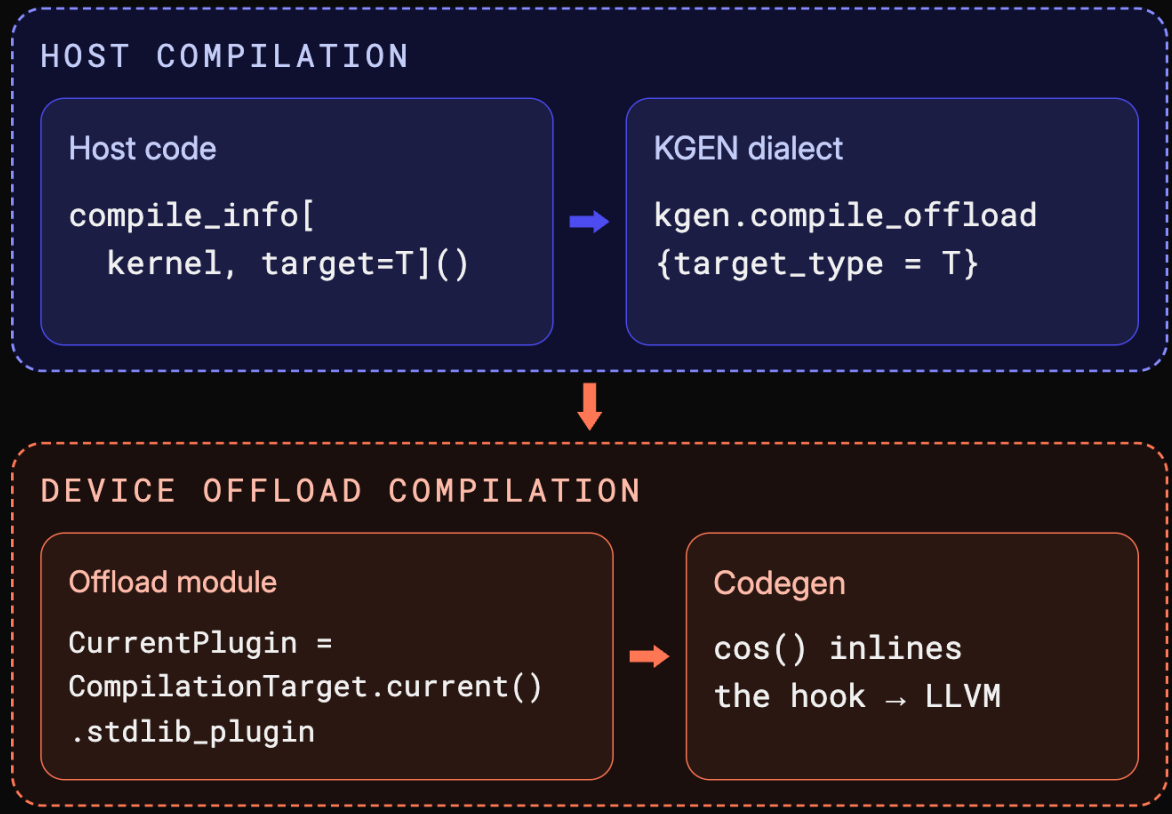
Choosing the stdlib plugin for an accelerator compile is as simple as passing the target parameter value.

Compilation targets pass from host to accelerator offload compilation

```mojo
def main():
    # x86_64-unknown-linux-gnu  (host)
    print(CompilationTarget.current())

    var k = compile_info[kernel, target=T]()
    launch(k)  # pseudocode

def kernel():
    # hexagon-unknown-linux-musl  (T)
    print(CompilationTarget.current())
```



Inside the offload module, CompilationTarget.current() is T, so T's plugin is the one that gets used.

Compiled kernels need a device to run on

The target flows through DeviceContext

```
struct DeviceContext[target: CompilationTarget = default_accelerator()]:
    def compile_function[func: ...](self) -> DeviceFunction[func, target=Self.target]:
        ...  # offload-compiles func for Self.target
```

Targets from any library plug into the Mojo driver

```
from qaic_std import default_qaic_accelerator

comptime target = default_qaic_accelerator()

var ctx = DeviceContext[target]()
var kernel = ctx.compile_function[vector_add]()   # QAIC target + QAICBackend hooks
```

When a custom compilation target is passed to DeviceContext, any driver plugins defined on it are usable too.

`DeviceContext[T]` → `compile_function[k]()` → `compile_info[k, target=T]` → `T.stdlib_plugin`

FUTURE

Launching it: an enqueue hook in MaxPluginHooks

```
trait MaxPluginHooks:
    comptime enqueue_function_fn: OptionalReg[_EnqueueFunctionFnType] = None
    # ... other hooks

struct DeviceContext[target: CompilationTarget]:
    def enqueue_function(self, f: DeviceFunction, *args, grid_dim, block_dim) raises:
        comptime if case .Some(launch) = MaxPluginForTarget[Self.target].enqueue_function_fn:
            return launch(self, f, args, grid_dim, block_dim)
        ...  # builtin CUDA / HIP / Metal launch path
```

Users write against one standard DeviceContext.enqueue_function API that works on any hardware implementing Modular's plugin hooks.

Accelerators that launch differently, like NPUs taking a packed argument array, plug in here.

More in Kern Handa's talk this afternoon.

IN PROGRESS

Two plugin .so files, found via environment variables

Driver plugin · run time · loaded by the Driver (HAL)

```
$ export MODULAR_DRIVER_PLUGINS="CUDA@/path/to/CUDAPlugin.so;CPU@/path/to/CPUPlugin.so"
```

;-separated name@path → dlopen + M_driver_* C ABI → DeviceContext picks the first entry matching its API

Compiler backend plugin · compile time · loaded by mojo

```
$ export MODULAR_COMPILER_PLUGINS="/path/to/libmojo-compiler-plugin.so"
```

;-separated .so paths → dlopen'd at compiler startup → modular.cfg plugin paths win, if set

FUTURE

A vendor plugin is "just" a directory

Every Modular plugin provides the same assets

```
vendor-plugin/
  targets.mojoc    # targets + PluginHooks
  compiler.so      # codegen, a compiler plugin
  driver.so        # runtime: next, with Manoj
```

The same plugin bundle will work with every layer of the Modular stack

```
$ mojo build app.mojo --plugin=vendor-plugin

$ max serve --model-path ... --plugin=vendor-plugin
```

Our goal is a world where new hardware support is boring and simple.

What could be simpler than a directory?

FUTURE

An ecosystem of plugin bundles, one set of interfaces

cuda-plugin/

```
compiler.so     # LLVM NVPTX
targets.mojoc   # PluginHooks + MaxPluginHooks
driver.so       # CUDA driver API
```

hip-plugin/

```
compiler.so     # LLVM AMDGPU
targets.mojoc   # PluginHooks + MaxPluginHooks
driver.so       # ROCm / HIP
```

qaic-plugin/

```
compiler.so     # LLVM Hexagon
targets.mojoc   # PluginHooks + MaxPluginHooks
driver.so       # Cloud AI SDK
```

tpu-plugin/

```
compiler.so     # MLIR Mosaic lowering
targets.mojoc   # PluginHooks + MaxPluginHooks
driver.so       # TPU runtime
```

Same three interfaces, every vendor:

Compiler: codegen backend | Mojo: PluginHooks / MaxPluginHooks | Driver: vendor SDK

Come build this ecosystem with us. Join the Modular alliance program: **modular.com/alliance**

Recap: target-neutral kernels, target-specific code

main.mojo

```
from my_plugin import T

def kernel(x, out):
  out[i] = exp(x[i])

ctx = DeviceContext[T]()
ctx.enqueue_function[kernel]()
```

mojo build

main

host binary
x86-64 / arm64

offloaded GPU assembly for T
7f45 4c46 0201 01…
baked in at build time, using T's hooks

run it

```
$ ./main   driver.so loaded
enqueue_function()
```

COMPILE TIME · THIS TALK

RUN TIME · MANOJ

The Modular Driver

LLVM Dev Meeting 2026

`Plugins for supporting execution of Mojo's compiled code`

Manoj Rajagopalan

Part 2 of 2, with Connor Gray

Modular

Outline

01 What is the Driver?

02 Architecture

03 HW-facing SDK API

04 How to build one

Typical kernel run

```
import max.sys

def main:

    var host: Host = max.sys.host()

    var num_devices: Int32 = host.device_count()

    var accelerator: Device = host.accelerator(0)  # GPU or Trainium etc.

    # More specific ways:

    # var cuda: Driver = host.backend("CUDA")  # optional: select your backend

    # var cuda_gpu: Device = cuda.device(0)
```

Typical kernel run (contd.)

```
var ctx: Context = accelerator.create_context()
var a_host: Buffer = ctx.create_buffer[dtype.Int32](NUM_ELEMS)  # blocking
populate(a_host)
var q: Queue = ctx.create_queue()
var a_gpu: Buffer = q.create_buffer[dtype.Int32](a_host.size) # non-block
q.copy(a_host, a_gpu)
q.execute[my_function](a_gpu) # compile and run
q.synchronize()
```

What is Modular's Driver?

API to support execution of Mojo code

Plugins for CUDA, HIP, QAIC, Metal, NRT, PjRT, CPU

`Bottom-most layer of the stack`

Mojo and Python surfaces: max.sys package

Plugin-friendly successor to DeviceContext

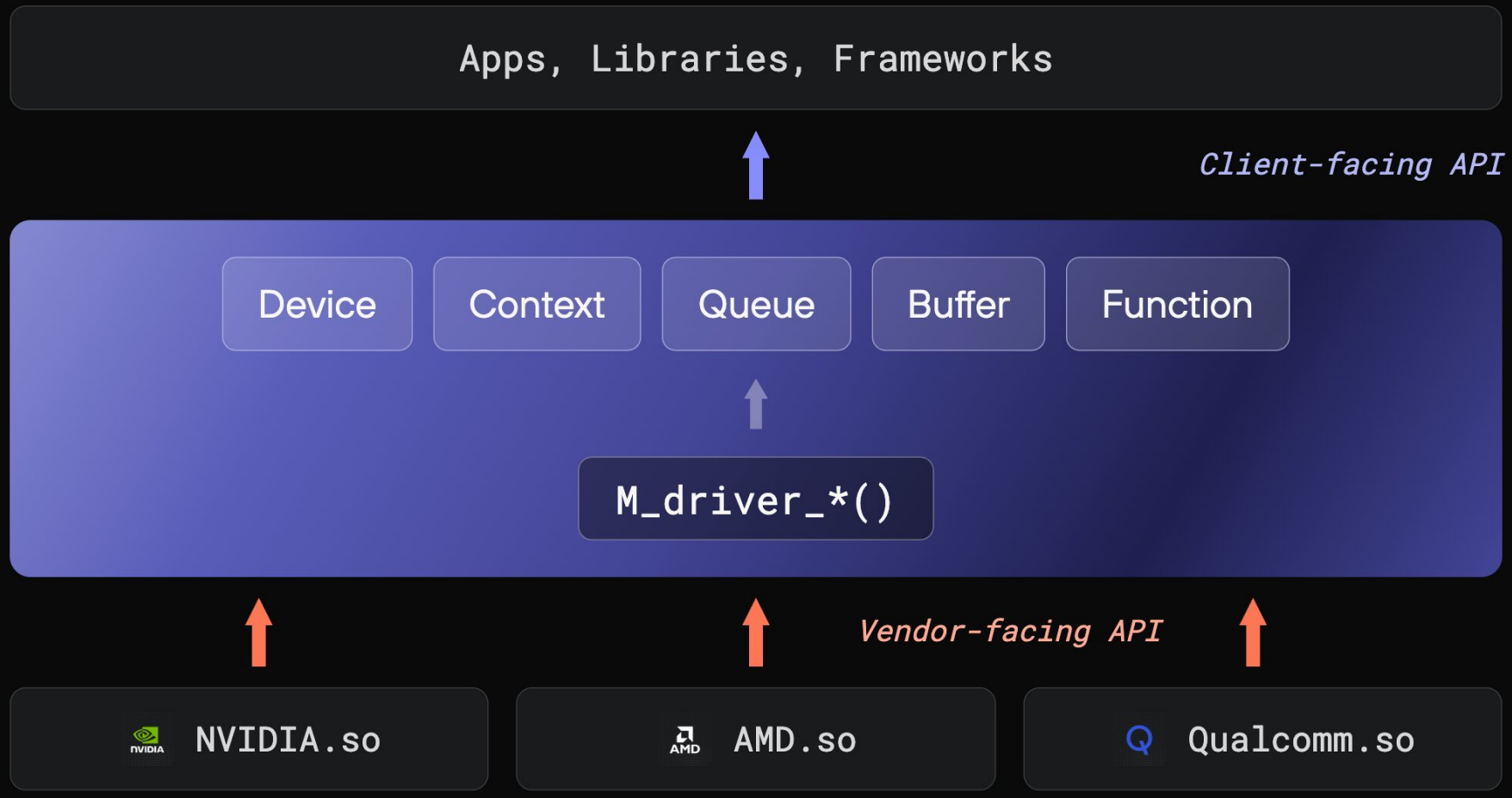
Many features shown are coming soon ...

Stratified API

Balance portability and performance:

- Hardware agnostic layer (HAL)*: full generalization*
- Capabilities*: partial generalization*
- Bespoke functions*: truly unique*

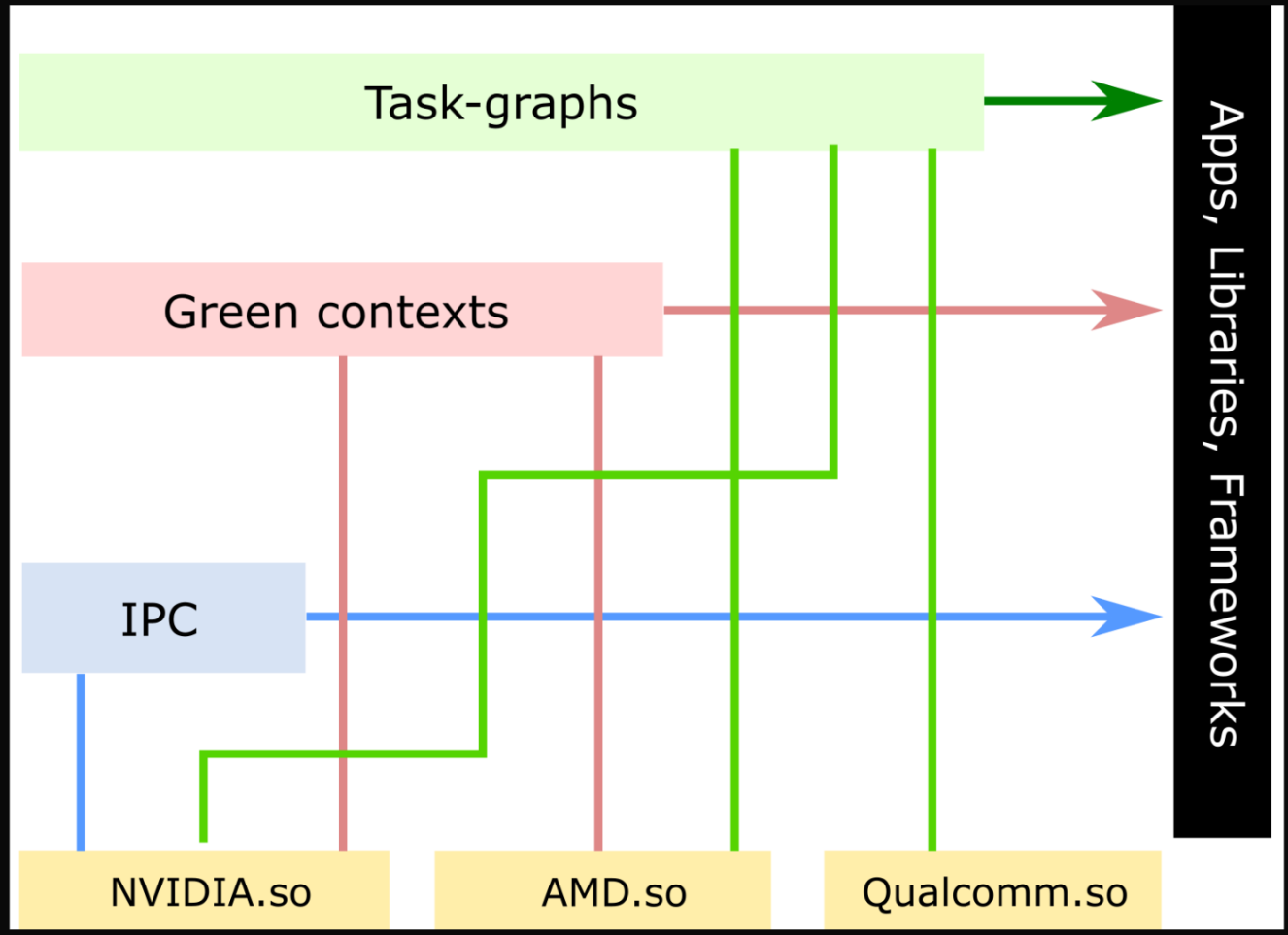
Architecture: Portable layer



Architecture: Performance layer

Named “capabilities”

Partially portable,
or completely specialized



HW-facing API (SDK)

```
typedef int64_t M_driver_status_t;


M_driver_create(version *version, driver_t *driver);
M_driver_device_get(driver_t driver, int64_t device_id, device_t *device);
M_driver_context_create(device_t device, context_t *ctx);
M_driver_queue_create(context_t ctx, queue_t *q);
M_driver_queue_copy_to_device(queue_t q, memory_view_t dst, const void *src);
M_driver_queue_execute(queue_t, function_t function,
                       execute_config_t *config, void **args,
                       uint64_t *arg_sizes, uint32_t num_args);
```

Building a Driver (plugin) in C/C++

1 Implement the vendor API using your HW driver/runtime:

```
#include "M_driver_api.h"  // Modular SDK API
#include "cuda.h"          // Your SDK

// Bridge code
M_driver_status_t M_driver_device_count(..., int64_t *count) {
    int cudaStatus = cuDeviceGetCount(count);
    return cudaStatus == CUDA_SUCCESS ? M_DRIVER_STATUS_SUCCESS
                                      : cudaToModularStatus(cudaStatus);
}
```

2 Build shared library:

```
g++ -fPIC -shared myDriverPlugin*.cpp -o myDriverPlugin.so
```

3 Install to file system as per prescribed layout in earlier slides

4 Invoke modular stack with env `MODULAR_DRIVER_PLUGINS=<abs path>/myDriverPlugin.so`

Questions?

Modular